

Normalizador estadístico vectorizado — NASM x86-64 + C

Proyecto de Arquitectura de Computadores enfocado en comparar una implementación **escalar** y una implementación **vectorizada con AVX2** de un normalizador estadístico.

El programa recibe un arreglo de números `float32`, calcula sus principales estadísticos y genera una versión normalizada mediante:

$$y[i] = \frac{x[i] - \mu}{\sigma}$$

La versión escalar procesa **1 elemento por iteración**, mientras que la versión vectorial procesa **8 elementos por iteración** utilizando registros YMM de 256 bits.

Funcionalidades implementadas

Ambas versiones implementan las funciones:

- `sum_array` : calcula la suma total del arreglo.
- `compute_stats` : calcula media, varianza poblacional, mínimo y máximo.
- `normalize_array` : genera el arreglo normalizado.

También se contemplan los casos borde principales:

- `N <= 0` .
- `N` no múltiplo de 8.
- desviación estándar igual a cero.
- procesamiento del remanente escalar en la versión AVX2.

Implementaciones

Versión escalar

Archivo:

```
asm/scalar/stats_scalar.asm
```

Características principales:

- NASM x86-64.
- Procesamiento de un `float32` por iteración.
- Uso de instrucciones escalares sobre registros XMM.
- Cálculo de media, varianza, mínimo y máximo.
- Normalización escalar.
- Convención de llamadas System V AMD64 ABI.

Versión vectorial

Archivo:

```
asm/vector/stats_vector.asm
```

Características principales:

- AVX2 con registros YMM de 256 bits.
- Procesamiento de 8 valores `float32` por iteración.
- Uso de `vbroadcastss`, `vsubps`, `vmulps`, `vaddps`, `vdivps`, `vminps` y `vmaxps`.
- Reducciones horizontales con `vextractf128`, `vhaddps` y operaciones de mezcla.
- Bucle escalar de cierre para el remanente cuando `N` no es múltiplo de 8.
- Uso de `vzeroupper` antes de retornar al código C.

Estructura del proyecto

```
.
├── Makefile
├── README.md
├── include/
│   └── stats.h
├── src/
│   └── driver.c
├── asm/
│   ├── scalar/
│   │   └── stats_scalar.asm
│   └── vector/
│       └── stats_vector.asm
├── tools/
│   ├── gen_input.py
│   └── verify_reference.py
├── diagramas/
│   ├── Diagrama 1.pdf
│   ├── Diagrama 2.pdf
│   ├── Diagrama 3.pdf
│   ├── Diagrama 4.pdf
│   ├── Diagramas_Arquitectura_Escalar_Vectorial.pdf
│   └── Enlaces Diagramas.md
└── data/
    └── archivos de entrada y salida generados durante las pruebas
```

Requisitos

- Linux.
- CPU con soporte AVX2.
- NASM.
- GCC.
- Make.
- Python 3.

- GDB.
- `perf` de forma opcional para análisis de rendimiento.

Para verificar soporte AVX2:

```
bash
lscpu | grep avx2
```

o:

```
bash
cat /proc/cpuinfo | grep avx2
```

Compilación

Desde la raíz del proyecto:

```
bash
make
```

Se generan los ejecutables:

```
bin/norm_scalar
bin/norm_vector
```

Ambos utilizan el mismo `driver.c`, pero enlazan con kernels diferentes.

Generación de datos de prueba

Ejemplo con un millón de elementos:

```
bash
python3 tools/gen_input.py 1000000 data/input.dat random
```

Otros ejemplos:

```
bash
python3 tools/gen_input.py 8      data/input_small.dat random
python3 tools/gen_input.py 1000  data/input_constant.dat constant
python3 tools/gen_input.py 0      data/input_empty.dat random
```

También deben probarse tamaños no múltiplos de 8, por ejemplo:

```
N = 7
N = 15
N = 1001
```

Esto permite comprobar el funcionamiento correcto del remanente escalar de la versión vectorial.

Ejecución

Versión escalar:

```
bash
./bin/norm_scalar data/input.dat data/output_scalar.dat 30
```

Versión vectorial:

```
bash
./bin/norm_vector data/input.dat data/output_vector.dat 30
```

El tercer argumento corresponde al número de repeticiones utilizadas para medir el tiempo promedio del kernel.

Durante la ejecución se obtienen los estadísticos calculados y el tiempo de ejecución. También se generan los archivos de salida normalizados y sus archivos de estadísticas.

Verificación de resultados

La correctitud puede verificarse con el script de referencia:

```
bash
python3 tools/verify_reference.py data/input.dat data/output_scalar.dat.stats.txt
```

y para la versión vectorial:

```
bash
python3 tools/verify_reference.py data/input.dat data/output_vector.dat.stats.txt
```

Las dos implementaciones deben producir resultados equivalentes dentro de la tolerancia definida para operaciones en `float32`.

Casos de prueba importantes

Se recomienda verificar como mínimo:

- `N = 0`.
- `N = 1`.
- `N = 7`.
- `N = 8`.
- `N = 15`.
- `N = 16`.
- arreglo con valores constantes.

- valores negativos.
- valores extremos.
- tamaños grandes para medición de rendimiento.

Depuración con GDB

Los binarios se compilan con símbolos de depuración, por lo que es posible inspeccionar directamente las funciones escritas en ensamblador.

Ejemplo:

```
bash
gdb --args ./bin/norm_vector data/input_small.dat data/out.dat 1
```

Dentro de GDB:

```
break normalize_array
run
info registers ymm0
stepi
```

Dependiendo de la versión de GDB también puede utilizarse:

```
print $ymm0.v8_float
```

o:

```
p/x $ymm0
```

Medición de rendimiento

El proyecto compara la implementación escalar y la vectorial utilizando mediciones repetidas del kernel.

El speedup se calcula como:

```
Speedup = tiempo_escalar / tiempo_vectorial
```

También puede utilizarse [perf](#) para observar ciclos, instrucciones y fallos de caché:

```
bash
perf stat -e cycles,instructions,cache-misses ./bin/norm_scalar data/input.dat data/output_scalar.dat 30
```

```
bash
perf stat -e cycles,instructions,cache-misses ./bin/norm_vector data/input.dat data/output_vector.dat 30
```

El objetivo es analizar por qué el speedup real puede ser menor que el máximo teórico de $8\times$ de AVX2, considerando factores como latencia, reducción horizontal, acceso a memoria y ancho de banda.

Diagramas

El repositorio incluye los cuatro diagramas requeridos para documentar la arquitectura de ambas implementaciones:

1. Arquitectura de software.
2. Flujo de control escalar y vectorial.
3. Asignación de registros.
4. Mapa de memoria y alineación.

También se incluye un PDF consolidado con los cuatro diagramas:

`diagramas/Diagramas_Arquitectura_Escalar_Vectorial.pdf`

Resumen

El proyecto implementa dos versiones funcionalmente equivalentes del mismo normalizador estadístico:

- **Escalar:** 1 `float32` por iteración.
- **Vectorial AVX2:** 8 `float32` por iteración más un bucle escalar para el remanente.

Esto permite comparar de manera directa la correctitud, la organización del código y el rendimiento obtenido mediante vectorización SIMD en x86-64.